Say Swear

Stage 2 Implementation Report

EECS 3311 - Fall 2026 | Status date: October 27, 2026 (Toronto)

Review draft - submission evidence is still pending. This report describes the public Stage 2 implementation. The user reports having played the game and confirms that it works properly. The instructor confirmed that the existing early commit history works for weekly-progress evidence. Videos/timestamps and focused source review remain pending; AI usage and cost figures are provisional student estimates where stated. Submission is due November 10, 2026.

This report accounts for all ten Stage 1 features, explains the five implemented patterns, and connects the current Java source to the revised UMLet diagrams. It separates recorded verification from evidence that still needs to be supplied.

Contents

1. Project overview and implementation status 2

2. Feature to implementation mapping 4

3. Implemented design patterns 8

4. Changes and rationale 12

5. AI tools and collaboration log 18

6. Development and verification summary 20

7. Demonstration index 23

Appendix A - UML class diagram 25

Appendix B - Use-case diagram 26

Appendix C - SD01 GUI voice command 27

Appendix D - SD02 CLI text command 28

Appendix E - SD03 contextual command and repeat 29

Appendix F - SD04 correction, combined and timed commands 30

Appendix G - SD05 fall and start recovery 31

1. Project overview and implementation status

Problem, users, and scope

Say Swear is a small 2D precision game for players who want to navigate a narrow winding path using natural-language instructions. One player character moves over a void toward one goal. Leaving the safe path cancels movement and returns the character to the level start. The implemented scope contains one fixed level, four-direction movement, combined directions, bounded timed movements, falling, and start recovery. There are no checkpoints, enemies, jumping, inventory, procedural generation, autonomous navigation, or pathfinding.

The human chooses every turn and its timing. A JavaFX GUI displays the actual level and accepts continuous microphone input, with a text-instruction field for the same interpreter. A Java terminal CLI accepts natural-language text and displays a map, coordinates, velocity, held directions, transcript/action feedback, and lifecycle. Both frontends use the same Java 21 controller, game, agent, and control objects composed by `Main.compose()`; the CLI substitutes typed text for capture/transcription.

AI integration and deterministic execution

GUI audio goes through `VoiceInputService` and `StreamingSpeechRecognizer`/`SherpaOnnxAdapter`. sherpa-onnx produces changing partial transcripts and endpoints. Audio is not streamed to SemIf. Eligible text and a frozen `ControlContext` go through `ApplicationController`, `VoiceCommandAgent`, and `DecisionModel`/`OpenJevAdapter` to the local SemIf service running Qwen3.5 4B. Python hosts that external inference runtime; the application, simulation, and frontends are Java.

SemIf scores finite alternatives from real model logits. The bridge composes one of 52 bounded semantic labels across eight action types, including `REPEAT_LAST`. The model receives transcript text, current held directions, prior accepted text, structured last movement, and movement eligibility. Java retains epoch/version/revision identity locally and stamps it on the returned decision. The request contains no map, position, route, start, or goal coordinates. The model interprets the human's command and selects a bounded intent. It does not directly call the game or choose how to reach the goal.

`ControlManager` checks schema, session epoch, transcript version, context revision, and player eligibility. It plans deterministic commands, cancels obsolete work, and executes through `ActionExecutor` and `GameCommand`. Java owns timer duration/expiry, repeat-memory resolution, conflict validation, simulation, collisions, observer cleanup, and fall recovery. `MovementIntent` remains available after a timed action releases its input, so a fresh `again` repeats the remembered movement rather than the release action.

The shared application loop serializes game/control changes. Worker threads handle microphone and HTTP work; JavaFX receives coalesced presentation updates. The current decision deadline is 2000 ms including agent queue time. New instructions supersede older requests; stale successes and failures cannot affect a newer session. The explicit `--demo` option uses a labelled deterministic parser for wiring checks. It is not an LLM and is never an automatic fallback.

Implementation and evidence status

Area	Current status	Evidence or remaining work
F01-F10 and both frontends	Implemented, with documented refinements	Complete feature/method/pattern mapping in section 2; genuine model and deterministic paths are distinguished.
Five course patterns	Implemented	Section 3 explains concrete participants and source methods; Appendix A contains the updated class diagram.
User gameplay	User-reported acceptance	The user states that they played the game and it works properly. Interface, microphone mode, individual scenarios, and code-review scope were not specified.
Ordinary Java verification	Recorded pass	Latest saved suite: October 19, 2026, 76 tests, zero failures/errors/skips. No application tests were rerun for report preparation.
Live and platform coverage	Partial recorded evidence	Native sample-audio ASR, historical real-model checks, and packaged CLI checks exist. Specific microphone/recovery, native Linux, and updated repeat-enabled 2000 ms GPU performance remain unverified.
AI tool accounting	Provisional student report	Approximately 6 billion Codex tokens and CAD $282.50 subscription spending are reported; local inference tokens and compute costs are unmeasured. Confirm period, currency, and allocation.
Weekly progress evidence	Instructor accepted existing history	Section 6 lists actual early code/test commits and the response to the student's early-completion question. Commit dates are unchanged.
Submission evidence	PENDING	All feature videos and detailed source-review outcomes remain to be supplied; AI accounting still needs final confirmation.

Reproduction and configuration

The repository includes a Gradle wrapper, Java dependencies, GUI/CLI launchers for Windows and Linux, and pinned setup instructions for SemIf and streaming ASR. Follow the root README, SemIf setup, and ASR setup. Start the real local service, wait for readiness, then launch GUI or CLI without demo mode. CLI play needs neither a microphone nor an ASR model.

Local inference requires no API key. `.env.example` contains safe environment-variable examples; the application does not auto-load `.env`. Configuration can be supplied through environment variables or launcher options. `.gitignore` excludes local environment files, downloaded weights, caches, logs, and build outputs. Report aggregate usage/cost totals with their accounting basis; private billing documents, account identifiers, and machine-specific paths are not needed.

Source references below are repository-relative paths. The supplied UMLet SVGs are preserved unchanged; their large-format PDF appendix pages allow readable zooming. Videos must eventually link to recordings accessible to the instructor and teaching assistants.

2. Feature to implementation mapping

All ten original feature IDs are accounted for. F10 replaces checkpoint recovery with return-to-start recovery. Repeat memory extends F05/F08/F09. Supporting input/session/display refinements are documented as changes, not counted as additional features.

Maintained feature mapping

This mapping follows the Course Project Stage 2 Instructions: every Stage 1 feature is accounted for with its status, implementation files/responsibilities, key methods, patterns, behavior type and video reference. The revised Stage 1 report retains F01-F10 and UC01-UC06. The original design is preserved in Git history at commit `56da5fd95c43dde4760032e59a36787a2c74049c`; current UMLet SVG diagrams describe the implementation. Design changes explain differences.

All ten features have connected implementations. **Implemented** does not imply a completed video or unrestricted reliability. The user reports having played the game and confirms that it works; no interface, microphone, command coverage or timing details were supplied. Specific microphone coverage and the updated 2000 ms GPU burst check remain unverified. Every demonstration is **PENDING - user recording, accessible link and timestamp required**.

Feature index

Feature	Stage 1 reference	Status/change	Type	Video
F01 - Real-Time Precision Movement	F01, same requirement	Implemented; CLI presentation refined	Deterministic	PENDING
F02 - Continuous Voice-Control Session	F02, same requirement	Implemented; lifetime/status refined	Deterministic session management	PENDING
F03 - Streaming Speech Transcription	F03, same requirement	Implemented	AI-based; Java delivers text	PENDING
F04 - Natural-Language Movement Interpretation	F04, same requirement	Implemented; local SemIf/Qwen provider	Hybrid	PENDING
F05 - Context-Aware Commands	F05, same requirement	Implemented; repeat memory added	Hybrid	PENDING
F06 - Live Command Correction / Interruption	F06, same requirement	Implemented; bounded scheduling	Hybrid	PENDING
F07 - Combined Directional Commands	F07, same requirement	Implemented	Hybrid	PENDING
F08 - Magnitude / Timed Commands	F08, same requirement	Implemented; timed replay added	Hybrid	PENDING
F09 - Control-State and Conflict Management	F09, same requirement	Implemented; freshness/replay guards	Deterministic	PENDING
F10 - Fall Detection and Start Recovery	F10 - Fall Detection and Checkpoint Recovery	Modified: every fall returns to start	Deterministic	PENDING

GUI text entry, explicit demo mode, session controls, repeat memory, terminal redraw and Linux setup are accounted for in D04-D07 and D18-D20. They extend these features or support setup and are not counted as additional features to reach ten. Checkpoint progression is explicitly removed under D17; no original feature ID is silently omitted.

F01 - Real-Time Precision Movement

Reference/status: Stage 1 F01; implemented, with D06/D10/D19 presentation/timestep refinements. **Design:** UC01/UC02; SD02/SD05. **Patterns:** MVC, Observer, State. **Type:** deterministic.

Files, responsibilities and key methods: ApplicationController schedules `tickFromClock()`/`advance()`. GameModel owns `update()`/`evaluatePosition()`. ActiveState subdivides `update()` for collision checks; Player applies `move()` and normalized velocity. Level supplies `isSafe()`/`isGoal()`. JavaFxGameView and CliView implement snapshot `render()`.

Input -> collaboration -> result: accepted directional controls set the game's held inputs. Loop ticks advance the player, check the corridor and publish snapshots. JavaFX draws the level/player; the CLI presents an adaptive map and exact position/velocity. Neither view simulates movement. Diagonal speed is normalized.

Verification: saved movement/geometry/goal tests and terminal tests pass; historical GUI/packaged CLI checks exercised actual movement. **Video: PENDING.** Show movement in both interfaces, a diagonal and state metrics.

F02 - Continuous Voice-Control Session

Reference/status: Stage 1 F02; implemented, refined by D11/D12. **Design:** UC01; SD01. **Patterns:** MVC, Adapter. **Type:** deterministic session management around ASR.

Files, responsibilities and key methods: JavaFxGameView routes `requestStartVoice()`/`requestStopVoice()`. ApplicationController coordinates `startVoiceControl()`, `stopVoiceControl()`, `handleVoiceReady()` and `handleVoiceFailure()`. VoiceInputService owns `start(epoch)`, capture-worker `run(session)`, `stop()` and `close()`. MicrophoneDevice opens Java Sound and supplies `readSamples()`, which the service's capture worker calls repeatedly.

Input -> collaboration -> result: Start listening creates a session and initializes recognition on a worker. After the microphone opens, readiness changes the visible status. Capture continues across utterances until stop/failure. Stop & release invalidates old work and releases controls/capture. CLI text substitutes for this audio entry point.

Verification: six controlled voice-service tests cover readiness, epoch isolation, nonblocking startup/stop, failure and PCM conversion. Specific real microphone session/recovery coverage remains unverified. **Video: PENDING.** Show starting, several utterances in one session, stopping and relevant unavailable-input behavior.

F03 - Streaming Speech Transcription

Reference/status: Stage 1 F03; implemented. **Design:** UC01; SD01. **Patterns:** Adapter, MVC. **Type:** AI-based recognition with deterministic delivery.

Files, responsibilities and key methods: StreamingSpeechRecognizer defines `start()`/`acceptAudio()`/`stop()`. SherpaOnnxAdapter loads the online model and decodes incrementally in `acceptAudio()`. VoiceInputService forwards `partial()` and advances utterance identity in `endpoint()`. TranscriptUpdate carries identity; ApplicationController.handleTranscript() versions submissions. JavaFxGameView.showTranscript() displays changing words.

Input -> collaboration -> result: successive audio chunks produce changed partial text before the utterance ends. The controller filters duplicates and submits current revisions. Endpoint detection starts a new utterance identity without requiring another recording. Transcript output is separate from the eventual accepted movement decision.

Verification: the historical native WAV check produced 17 partials, one endpoint, expected phrases and no errors; it opened no microphone. Live speech accuracy/latency remain unmeasured. **Video: PENDING.** Show partial words changing during speech, then the separate action feedback.

F04 - Natural-Language Movement Interpretation

Reference/status: Stage 1 F04; implemented with local SemIf/Qwen instead of the original provider assumption (D01/D13). **Design:** UC01/UC02; SD01/SD02. **Patterns:** Adapter, Command, MVC. **Type:** hybrid.

Files, responsibilities and key methods: VoiceCommandAgent.interpret() schedules the DecisionModel.decide() contract. OpenJevAdapter submits HTTP in `decide()` and validates the allowlisted output in `parseResponse()`. The SemIf bridge runs `DecisionEngine.decide()` with genuine Qwen logits. ControlManager.accept()/plan() creates commands; ActionExecutor.execute() applies them.

Input -> collaboration -> result: spoken text or typed `go up` reaches the same agent. Qwen chooses a bounded semantic action; Java stamps trusted identity, validates it and changes the game through commands. The view shows accepted type/directions or no-action/error feedback. The model never calls game methods. `--demo` explicitly selects a deterministic parser and is not the AI implementation.

Verification: recorded genuine-model and packaged CLI checks exercised the complete path. Finite samples do not guarantee every paraphrase. Current input errors stop controls; an unclear `NO_ACTION` leaves existing movement unchanged. **Video: PENDING.** Use the real model for direction/stop and an unclear or unsupported instruction.

F05 - Context-Aware Commands

Reference/status: Stage 1 F05; implemented and extended with repeat memory (D18). **Design:** UC03; SD03. **Patterns:** Adapter, Command. **Type:** hybrid.

Files, responsibilities and key methods: ControlContext stores held directions, prior accepted text and MovementIntent. VoiceCommandAgent.interpret() freezes one utterance's baseline; OpenJevAdapter.decide() sends it to Qwen. ControlManager.getContext()/accept()/plan() maintain/replay memory; ApplicationController.acceptanceMessage() explains missing history.

Input -> collaboration -> result: `other way` uses initially held directions; `keep going` preserves movement/timer. After timed movement expires, `again` asks Qwen for `REPEAT_LAST`; Java creates fresh commands from the stored direction/duration. With no history, repeat changes nothing and explains why. The model receives no map or goal.

Verification: frozen-context/history/no-history tests pass; eight recorded CPU repeat/history samples passed under a diagnostic deadline. Updated default-budget GPU verification is pending. **Video: PENDING.** Demonstrate reversal, keeping movement, repeat after expiry and repeat after restart without history.

F06 - Live Command Correction / Interruption

Reference/status: Stage 1 F06; implemented with D11/D12/D15/D16 refinements. **Design:** UC04; SD04. **Patterns:** Adapter, Command. **Type:** hybrid.

Files, responsibilities and key methods: ApplicationController.handleTranscript()/submitTextCommand() create newer versions. VoiceCommandAgent.interpret()/reset() bound work to two active calls and one latest pending input. ControlManager.beginInput()/accept()/plan() reject stale results and prepare replacements; ActionExecutor.cancelAll()/execute() replace active timed work.

Input -> collaboration -> result: `right - no, left` produces revised text. Qwen interprets the correction; only a matching version/epoch/control revision may act. Java cancels old timed work before applying replacement directions. A typed instruction supersedes remaining partials of an older spoken utterance. Cancellation cannot later release a newer command.

Verification: stale-result/replacement tests pass. The historical pre-repeat four-partial GPU burst returned LEFT after 990.7 ms within 2000 ms; this is not current repeat-schema timing evidence. Specific microphone correction coverage is unverified. **Video: PENDING.** Correct active movement and interrupt a timed action.

F07 - Combined Directional Commands

Reference/status: Stage 1 F07; implemented. D13 describes axis readouts. **Design:** UC05; SD04. **Patterns:** Adapter, Command. **Type:** hybrid.

Files, responsibilities and key methods: DecisionEngine.decide() obtains model-selected semantic directions; OpenJevAdapter.decide() transmits the request and validates the returned choice. ControlManager.validate()/plan() enforce a compatible set. KeyDownCommand.execute() calls `GameModel.press()`; Player.refreshVelocity() normalizes speed.

Input -> collaboration -> result: `up and right` selects both axes. Java releases unwanted old inputs, presses the desired pair and displays diagonal movement/held directions. Opposing pairs are excluded from the provider vocabulary and rejected by Java validation.

Verification: compatibility/normalized-speed tests pass; recorded model samples covered all four perpendicular pairs. **Video: PENDING.** Show combined movement and the actual response to a contradictory instruction; do not claim universal language accuracy.

F08 - Magnitude / Timed Commands

Reference/status: Stage 1 F08; implemented with timed replay under D18. **Design:** UC05/UC03; SD04/SD03. **Patterns:** Adapter, Command. **Type:** hybrid.

Files, responsibilities and key methods: OpenJevAdapter.decide() limits semantic durations to 100/200/400 ms. ControlManager.validate()/plan()/advance() validate/create commands and change revision at expiry. TimedPressCommand.execute()/advance()/cancel() own duration; ActionExecutor.advance() advances/removes timers.

Input -> collaboration -> result: `a little left` selects a 200 ms hold. Java presses, then releases the direction automatically. Equivalent partials within that utterance cannot extend its timer. A fresh `again` creates a new timed command from remembered semantics. Timing uses the simulation loop, not independent delayed callbacks.

Verification: expiry, quantization, interruption and repeat tests pass. Packaged live CPU CLI replay used a 30000 ms diagnostic interpretation budget; the movement hold remained 200 ms of simulation time. **Video: PENDING.** Show temporary movement, automatic release and fresh repeat after expiry.

F09 - Control-State and Conflict Management

Reference/status: Stage 1 F09; implemented with failure/replay refinements. **Design:** UC01-UC06; SD01/SD03/SD04/SD05. **Patterns:** Command. **Type:** deterministic.

Files, responsibilities and key methods: GameModel owns held inputs. ControlManager.beginInput()/accept()/validate()/reset() own version/epoch/revision and memory. ActionDecision.withRequest() receives local metadata from VoiceCommandAgent. ActionExecutor.cancelAll(), ReleaseAllCommand.execute() and ApplicationController.failCurrent() neutralize current failures.

Input -> collaboration -> result: each proposal is checked against current transcript/session/control revision before execution. Invalid current actions and current timeouts release controls; stale responses/failures cannot alter a newer session. Duplicate partials cannot rearm timers/repeats. Pause/fall/restart/current failure clear memory; ordinary stop, expiry and keep/no-action preserve it.

Verification: tests cover malformed/stale proposals, opposite directions, duplicate timers, expiry revision, reset/history and late failures. **Video: PENDING.** Show held-state feedback, interruption and current unavailable-service cleanup; explain the freshness guard without substituting mocked AI for real behavior.

F10 - Fall Detection and Start Recovery

Reference/status: original F10 - Fall Detection and Checkpoint Recovery; modified and implemented. D17 removes checkpoint progression. **Design:** UC06; SD05. **Patterns:** State, Observer, Command. **Type:** deterministic.

Files, responsibilities and key methods: GameModel.evaluatePosition()/resetToStart() detect the unsafe footprint and restore `Level.start()`. FallingState.update() initiates reset; RespawningState.update() restores active play. ApplicationController.onGameEvent()/invalidateControls() invalidate AI work and controls; GameView.onGameEvent() presents lifecycle feedback.

Input -> collaboration -> result: human-directed movement leaves the corridor. FallingState and the fall event trigger epoch invalidation, interpretation/timer cancellation and release. Later lifecycle ticks restore start position and zero velocity, then ActiveState. If listening was requested, voice capture restarts; movement requires fresh input. This deliberately changes recovery from preserving checkpoint progress to losing all progress on a fall.

Verification: saved tests check repeated falls after level progress return to start and reject pre-fall responses. Specific real microphone restart/recovery coverage remains unverified. **Video: PENDING.** Show a fall, neutral recovery at start and fresh movement afterward.

3. Implemented design patterns

Maintained pattern explanations

The five patterns proposed in Stage 1 remain implemented: **MVC, Command, Adapter, Observer and State**. Each solves a concrete problem in the shared Java application. The change record explains implementation refinements; no pattern has been replaced and no repeated use is counted as an additional pattern.

Current implementation UML

Diagram source: Implemented class diagram (embedded in the appendix).

The current class diagram is a UMLet-exported SVG. It shows the shared GUI/CLI architecture, control epoch/version/revision fields, `ActionExecutor.activeTimed`, `TimedPressCommand.remainingMs`, structured movement memory and the external SemIf boundary. Its historical "Stage 1 revised" title identifies the design lineage; the classes and key operations describe the current implementation. Concrete classes inherit interface operations, so inherited methods need not be repeated in each box.

The use-case diagram and affected sequences are also current: SD01 voice, SD02 CLI, SD03 context/repeat, SD04 correction/timed actions, and SD05 start recovery. Original diagram sources/exports remain in Git history; they are not claimed to exist as current repository files.

1. MVC - shared behavior for two frontends

Problem: JavaFX and the terminal need different rendering/input mechanisms while exposing the same game and agent functionality. Duplicating movement, interpretation or recovery in the views would make behavior diverge.

Participant and source	Role and actual methods
GameModel, Player, Level	Model: authoritative held inputs, player state and geometry. `update()`, `getSnapshot()`, `evaluatePosition()`, `move()`, `isSafe()`, `isGoal()`.
ApplicationController	Controller: serializes input, decision acceptance, simulation and cleanup. `submitTextCommand()`, `handleTranscript()`, `handleDecision()`, `advance()`, `onGameEvent()`.
GameView	Presentation contract: `render()`, `showTranscript()`, `showVoiceStatus()`, `showMessage()`.
JavaFxGameView, CliView	Views: draw snapshots and present feedback. GUI `submitTypedCommand()`/`requestStartVoice()` route intent to the controller; CLI `readCommand()` returns text for the launcher to submit.
Main	Composition: `compose()` builds the shared core for either frontend; `runCli()` handles terminal session commands.

How it follows the pattern: a GUI text submission or CLI instruction reaches the same controller and agent. Accepted commands mutate `GameModel`; immutable snapshots reach the selected view. JavaFX scheduling and JLine terminal redraw remain presentation concerns. Neither renderer advances player physics or interprets direction language.

Why useful / without it: fixes to collision, correction or repeat behavior apply to both frontends. Without this separation, consistent GUI/CLI tests and alternate presentation would require changes to duplicated logic.

Stage 1 comparison: MVC is retained. D03 makes `GameView` inherit the observer contract, D04 adds GUI typed input, and D06/D19 refine rendering. These change presentation plumbing rather than replacing MVC. Features: F01-F04 and shared presentation of F05-F10.

2. Command - cancellable deterministic movement

Problem: a new correction, fall or timeout must interrupt old timed movement without a delayed release affecting a newer command. AI output must not directly mutate game state.

Participant and source	Role and actual methods
GameCommand	Command interface: `execute(GameModel)` and `cancel(GameModel)`.
KeyDownCommand, KeyUpCommand	Concrete commands: press or release one direction through `execute()`; cancellation never reintroduces an already released input.
TimedPressCommand	Concrete timed command: `execute()`, `advance()`, `cancel()` own one compatible direction set and its remaining duration.
ReleaseAllCommand	Concrete neutralizing command: `execute()` clears held controls.
ControlManager	Client/planner: `accept()`, `validate()` and `plan()` translate a fresh semantic decision into command objects.
ActionExecutor	Invoker: `execute()`, `cancelAll()`, `advance()` own the active timed-command list and rollback partial failures.
GameModel	Receiver: `press()`, `release()`, `releaseAll()` update authoritative inputs.

How it follows the pattern: a `SWITCH LEFT` proposal becomes release-right and press-left commands. The manager computes the desired set, cancels old timed work, then the executor applies the replacement on the application loop. Timer expiry is advanced on that same loop; there is no detached delayed-release callback. A `REPEAT_LAST` creates fresh commands from remembered semantics, never reuses an old command object or request identity.

Why useful / without it: cancellation, timed behavior and failure rollback are testable without an LLM. Without explicit command objects, these responsibilities would be spread across interpretation callbacks and game mutation code, making stale timer effects harder to prevent.

Stage 1 comparison: the planned pattern/classes remain. D10 specifies elapsed-time advancement; D11 centralizes invalidation; D18 adds fresh replay through the existing command mechanism. No general undo/history system is claimed. Features: F04-F10 where decisions, interruption or recovery require commands.

3. Adapter - isolate speech and decision providers

Problem: native sherpa-onnx APIs and SemIf's local HTTP/logits service have different input, lifetime and failure conventions. Those details must not spread into game/control logic.

Participant and source	Role and actual methods
StreamingSpeechRecognizer	Target interface: `start(onPartial, onEndpoint, onError)`, `acceptAudio()`, `stop()`, `close()`.
SherpaOnnxAdapter	Adapter: creates the native online recognizer/stream, feeds audio, converts partials/endpoints and releases native resources. `start()`, `acceptAudio()`, `stop()`.
sherpa-onnx `OnlineRecognizer` / `OnlineStream`	External adaptees, used through the adapter's typed Java API.
DecisionModel	Target interface: asynchronous `decide(TranscriptUpdate, ControlContext)` returning `ActionDecision`.
OpenJevAdapter	Adapter: `decide()` builds a control-only request; `parseResponse()` checks backend/model and maps an allowed label to semantic output.
SemIf bridge and pinned Qwen3.5 4B	External adaptee boundary: `/decide` uses genuine SemIf logits scoring and returns a bounded choice. The bridge is supporting model-runtime code; the application remains Java.
VoiceInputService, VoiceCommandAgent	Clients depend on the target interfaces, allowing controlled substitutes in tests.

How it follows the pattern: speech callbacks become provider-independent transcripts; HTTP/scoring output becomes a provider-independent semantic decision. Neither adapter calls `GameModel.press()` or

performs route planning. The optional DemoDecisionModel implements the same target for explicitly labeled offline use; it is not the real LLM integration or an automatic fallback.

Why useful / without it: native deployment and provider protocol changes can be isolated and tested. Without adapters, the controller would need native recognition and HTTP/schema details, and testing failures would require live hardware/services.

Stage 1 comparison: the same two target interfaces/adapters remain. D01 corrects the provider identity to SemIf/Qwen, D13/D16 refine bounded readouts, and D14 specifies native deployment. These are adaptee/integration changes, not a new counted pattern. Features: F02-F08.

4. Observer - lifecycle cleanup and presentation

Problem: a fall or goal must notify both cleanup/control coordination and presentation without the game depending on a particular GUI or terminal class.

Participant and source	Role and actual methods
GameModel	Subject: `addObserver()`, `removeObserver()`, `notifyObservers()` publish domain events synchronously.
GameObserver	Observer interface: `onGameEvent(GameEvent)`.
GameEvent, GameEventType	Event value/type: state change, fall, respawn and goal, with an immutable snapshot.
ApplicationController	Concrete observer: `start()` registers cleanup first; `onGameEvent()` invalidates controls/agent work and coordinates voice recovery.
GameView, JavaFxGameView, CliView	Presentation observers: inherited/default or overridden `onGameEvent()` renders snapshots and lifecycle messages; CLI throttles ordinary refresh while preserving lifecycle output.

How it follows the pattern: `GameModel.evaluatePosition()` publishes `FALL_DETECTED`; the controller cancels old work, and the selected view presents the fall. Respawn/goal notifications follow the same contract. The game refers to `GameObserver`, not frontend implementations. Notification uses a copied subscriber list and isolates an observer's runtime exception so other subscribers still receive the event.

Why useful / without it: lifecycle cleanup and rendering can evolve independently. Without observer notification, the game would need direct controller/view calls or duplicated polling logic. Synchronous ordering here is deliberate, allowing cleanup before presentation observers.

Stage 1 comparison: the planned subject/observers remain. D03 centralizes the view contract, D08 adds removal/exception isolation, and D17 removes checkpoint events as part of the documented gameplay change. Features: F01/F10 and lifecycle control cleanup supporting F02/F09.

5. State - explicit player lifecycle

Problem: movement, recovery and goal completion need different behavior. Repeating lifecycle conditionals across game updates and frontends would make it easy to move during recovery or after finishing.

Participant and source	Role and actual methods
Player	Context: stores `PlayerState`; `update()` delegates, `setState()` changes behavior and stops velocity when movement is disallowed.
PlayerState	State interface: `update(player, game, deltaTime)`, `allowsMovement()`, `status()`.
ActiveState	Concrete active state: subdivides movement and evaluates position in `update()`; permits movement.
FallingState	Concrete falling state: disallows movement; its next `update()` calls `GameModel.resetToStart()`.
RespawningState	Concrete recovery state: next `update()` restores `ActiveState` and publishes `RESPAWNED`.
FinishedState	Concrete completed state: disallows movement and keeps velocity zero in `update()`.

Participant and source	Role and actual methods
GameModel	Transition collaborator: `evaluatePosition()`, `resetToStart()`, `restart()` select lifecycle transitions.

How it follows the pattern: the normal path is Active -> Falling -> Respawning -> Active, or Active -> Finished. These are different state objects with behavior, not only an enum with scattered branches. `PlayerStatus` is the immutable presentation value; it does not replace polymorphic state execution.

Why useful / without it: each lifecycle's update policy can be tested in isolation, and frontends receive the same status. Without State, movement and recovery rules would become repeated conditional checks and could disagree across paths.

Stage 1 comparison: all four planned states remain. D02 adds a typed status enum; D17 changes the recovery destination from a saved checkpoint to the level start. The State pattern still implements the same lifecycle separation. Features: F01/F10, with lifecycle gating used by F09.

Verification boundary

The saved ordinary Java suite exercises game states, observer subscription, cancellation/rollback, freshness, both presentation contracts and provider boundaries. Recorded real-model and native-ASR checks are separate from controlled tests. The user has given general gameplay acceptance; specific microphone coverage and the updated default-budget GPU gate remain unverified. See validation for dates, commands and limitations, and the feature mapping for pending video references.

Current UML and implementation contracts

The class model shows `ActionDecision`/`ControlContext` epoch and revision metadata, `ControlManager` freshness state, `ActionExecutor.activeTimed`, and the remaining duration owned by `TimedPressCommand`. These make concurrency and timer ownership structurally visible, as suggested by the Stage 1 feedback. F02/F03/F04 have separate acceptance criteria in sections 2 and 7.

Diagram	Current UMLet SVG	PDF appendix
Class model	class-diagram.svg	A
Use cases UC01-UC06	use-case-diagram.svg	B
SD01 - GUI voice command	sd01-gui-voice-command.svg	C
SD02 - CLI text command	sd02-cli-text-command.svg	D
SD03 - Contextual command and repeat	sd03-contextual-command.svg	E
SD04 - Correction, combined and timed actions	sd04-live-correction.svg	F
SD05 - Fall and start recovery	sd05-fall-recovery.svg	G

4. Changes and rationale

The original Stage 1 snapshot is retained in Git history at `56da5fd95c43dde4760032e59a36787a2c74049c`. The revised design and current diagrams reflect the implemented local-model, repeat-memory, start-recovery, and terminal-display contracts. The following record explains changes against the earlier baseline rather than treating the revised design as proof that no changes occurred.

Maintained change record

This is the consolidated **Original design -> Implemented change -> Reason -> Impact** record required by the Course Project Stage 2 Instructions. F01-F10 remain traceable through the feature mapping. The same five patterns remain implemented; pattern roles and methods explain their structural realization.

Baseline, current artifacts and evidence

The original Stage 1 design is preserved in Git history at commit `56da5fd95c43dde4760032e59a36787a2c74049c`. The revised Stage 1 report and current UMLet-exported SVGs describe the selected implementation. Earlier PlantUML sources, rendered previews and `docs/uml/stage1/` archives mentioned in development records are historical artifacts in Git history, not current-file promises. UMLet replaced the diagram authoring/export format; this did not change application behavior or add a design pattern.

Recorded verification progressed from 63 ordinary Java tests, to 73 after repeat support, to 76 after terminal redraw. Nine controlled bridge tests and 28 CPU semantic samples were recorded for repeat support. Earlier 20-case GPU and four-partial burst checks predate repeat. Historical diagram rendering/geometry/hash checks apply to the versions named in validation, not automatically to replacement SVGs.

The user reports having played the game and confirms that it works. This is general gameplay acceptance; no interface, microphone, command coverage or timing details were supplied. Specific microphone coverage and the updated 2000 ms GPU burst check remain unverified. Videos remain pending. These limits do not imply that the corresponding code is missing.

D01 - Correct the decision provider identity

Original: the initial report interpreted OpenJev as the unrelated hosted `openjev.sh` service with an `openjev` model alias.

Implemented change: retain Java `DecisionModel`/`OpenJevAdapter`, but connect to a project loopback HTTP bridge for genuine **SemIf (formerly OpenJev), using Qwen3.5 4B**. The upstream SemIf commit is pinned to `23cf1f39fc9534fe81437200959b6dfc7106e45a`. Python hosts the external inference dependency; the game, frontends and controls remain Java.

Reason / original need: correct the provider research error after the user identified the intended project and model. Natural-language movement interpretation remains the original user need; it now runs locally without an API key.

Impact: F04-F08; UC01-UC05; Adapter preserved. OpenJevAdapter.decide()/parseResponse(), DecisionModel.decide(), and DecisionEngine.decide()/load_scorer() implement the boundary. Genuine-model and packaged CLI checks are recorded separately from demo tests.

D02 - Concrete immutable values and typed status

Original: value objects were conceptual and snapshot lifecycle status was string-valued.

Implemented change: use records for `Vector2`, `GameSnapshot`, `GameEvent` and agent/control values, plus `PlayerStatus`. Copy direction sets and level geometry into immutable collections.

Reason / original need: safely publish one model to both views and avoid spelling-dependent lifecycle handling. Movement and recovery behavior remain unchanged.

Impact: F01/F05/F09/F10; UC01-UC06; MVC, Observer and State preserved. GameSnapshot constructor, ControlContext constructor, Level constructor, Player.status(), PlayerStatus define the values. The enum is for presentation; `PlayerState` objects still execute lifecycle behavior.

D03 - One presentation/observer contract

Original: both concrete views separately implemented `GameView` and `GameObserver`; transcript/status methods were frontend-specific.

Implemented change: `GameView` extends `GameObserver`, with default transcript/status hooks and lifecycle presentation. Concrete views provide rendering and messages; CLI overrides event handling for its output policy.

Reason / original need: give the controller one presentation contract and centralize common fall/respawn/goal messages without duplicating the simulation.

Impact: F01-F04/F10; UC01/UC02/UC06; MVC and Observer refined, not replaced. GameView.onGameEvent()/render()/showMessage(), CliView.onGameEvent(), ApplicationController.start() implement the change.

D04 - Add GUI natural-language text input

Original: GUI instructions entered through speech; CLI instructions were typed.

Implemented change: add the GUI Text Instructions field and Send instruction action, routed to the same text-command controller method used by CLI.

Reason / original need: allow users to exercise/correct movement when audio setup is unavailable. This is an added entry route for F04, not direct keyboard movement or a substitute for F02/F03.

Impact: F04-F09; UC01/UC02/UC04; MVC preserved. JavaFxGameView.submitTypedCommand(), ApplicationController.submitTextCommand(), Main.runCli() share the pipeline. Historical GUI smoke testing exercised actual text-field movement with explicit demo mode.

D05 - Add fresh-run and inspection controls

Original: session start/stop and checkpoint recovery were planned; a manual fresh-run control was not specified.

Implemented change: GUI Restart run and CLI `/restart`, `/state`, `/pause`, `/help`, `/quit` provide deterministic session operations. Restart invalidates pending work before resetting the game.

Reason / original need: let users restart the single level, inspect exact state and stop a session. These supporting operations are not counted toward the ten meaningful features.

Impact: F01/F02/F09/F10; UC01/UC02/UC06; MVC, Command, Observer and State preserved. ApplicationController.restart()/pauseControls(), GameModel.restart(), Main.runCli() and the GUI button handlers implement them. GUI and packaged CLI checks covered reset/release behavior.

D06 - Implement responsive presentation scheduling

Original: snapshots and throttled CLI display were described without concrete scheduling utilities.

Implemented change: JavaFX coalesces snapshots/transcript/messages on its application thread. The initial CLI used a 61 x 27 ASCII overview and approximately one-second refresh; explicit quiet mode suppressed periodic scripted output. D19 later replaces the scrolling interactive presentation. The current GUI also highlights the neutral player at the start; this is presentation of existing state.

Reason / original need: prevent repaint accumulation and make the player/state readable without adding game mechanics. At the initial stage, Gradle-forwarded streams made `System.console()` alone insufficient to identify a usable terminal.

Impact: F01/F03/F04/F10; UC01/UC02/UC06; MVC and Observer preserved. JavaFxGameView.render()/scheduleUpdate()/drainUpdates(), CliView.formatSnapshot()/onGameEvent() are relevant. D19 supersedes the original append-only CLI refresh policy, not the shared model.

D07 - Add an explicitly selected offline demo model

Original: semantic interpretation used the configured real `DecisionModel`.

Implemented change: add `DemoDecisionModel`, a limited deterministic parser selected only by explicit `--demo`. Real SemIf remains the default; there is no automatic fallback to demo when inference fails.

Reason / original need: permit application development/demonstration without loading an AI service. This does not replace the required LLM integration or establish AI accuracy.

Impact: F04-F09 development/demo path; UC01-UC05; Adapter target contract preserved. DemoDecisionModel.decide()/parse(), Main.compose(), SaySwearApplication.start() select/label the mode. Demo results are identified separately in validation.

D08 - Define observer removal and failure isolation

Original: subject notification was planned without a subscriber-removal or exception policy.

Implemented change: notify a copied subscriber list, allow removal, and catch/log individual subscriber failures while continuing notification. Register controller cleanup before the presentation observer.

Reason / original need: a presentation failure must not prevent lifecycle cleanup; close/unsubscribe must be safe during notification.

Impact: F01/F09/F10; UC01/UC02/UC06; Observer refined. GameModel.removeObserver()/notifyObservers(), ApplicationController.start()/close() implement the policy. Observer removal and fall cleanup have deterministic tests.

D09 - Choose a reproducible Java build

Original: Java/JavaFX were required without concrete build and dependency versions.

Implemented change: use Java 21, JavaFX 21, Gradle wrapper 8.14.2, pinned sherpa Java/native dependencies and JUnit. Platform classifiers select native dependencies for the host.

Reason / original need: provide reproducible compilation, ordinary tests and packaged GUI/CLI startup rather than requiring manually assembled classpaths.

Impact: all features and UC01-UC06; all five patterns retained, no added pattern. build.gradle defines `test`, `installDist`, `guiSmokeTest`, `asrSmokeTest` and `liveSemIfTest`; wrapper configuration pins Gradle. Historical suites increased 63 -> 73 -> 76 ordinary tests as functionality evolved; special tasks remain opt-in. D20 records Linux setup entry points.

D10 - Specify elapsed-time simulation and timer ownership

Original: a fixed 60 Hz timestep and roughly one-tick timer quantization were proposed.

Implemented change: schedule updates every 16 ms, measure elapsed time and cap one update at 50 ms. Advance timers before game movement; subdivide movement for corridor collision checks. `ActionExecutor` owns active timed commands and `TimedPressCommand` owns remaining duration.

Reason / original need: bound motion after a delayed loop turn and prevent a large update from skipping across void. Timer tolerance follows the actual capped update; prolonged stalls can make simulation lag wall time.

Impact: F01/F08/F09/F10; UC01/UC02/UC05/UC06; Command and State preserved. ApplicationController.tickFromClock()/advance(), ActionExecutor.advance(), TimedPressCommand.advance(), ActiveState.update() implement this. Movement, quantization and collision tests cover deterministic behavior.

D11 - Centralize request invalidation and cleanup

Original: sequences used explanatory operations `handleInputFailure(message, epoch, version)` and `invalidateControls(reason)`.

Implemented change: completion callbacks check captured identity and use private `failCurrent(message)`/`invalidateControls()` helpers. Explicit `close()` lifecycles release resources. Control reset advances the epoch before cancellation callbacks run; decisions carry local version/epoch/revision metadata.

Reason / original need: use one neutral cleanup path and prevent old successes/failures from changing a newer input session. Public implementation operations differ from the conceptual signatures without changing their responsibility.

Impact: F02/F04/F06/F09/F10; UC01/UC02/UC04/UC06; MVC, Command, Adapter and Observer preserved. ApplicationController.isCurrent()/failCurrent()/invalidateControls()/close(), ControlManager.reset()/accept(), VoiceCommandAgent.reset()/close(), VoiceInputService.stop()/close() implement cleanup. Tests cover stale

callbacks and bounded/reset behavior.

D12 - Define precedence between voice and typed input

Original: the selected frontend supplied voice or typed text, so simultaneous input-source precedence was unspecified.

Implemented change: a typed command supersedes remaining partials of the current spoken utterance. Fresh voice becomes eligible after a new endpoint. Both sources share one version counter and control pipeline.

Reason / original need: prevent an older evolving transcript from undoing a newer typed correction after D04 added GUI text entry.

Impact: F02/F03/F06/F09; UC01/UC02/UC04; MVC and Adapter preserved. ApplicationController.submitTextCommand()/handleTranscript() and VoiceInputService.endpoint() implement source/utterance identity. Specific mixed microphone/text coverage remains unverified.

D13 - Factor bounded semantic selection into model readouts

Original: the adapter conceptually obtained one bounded action from the provider.

Implemented change: genuine upstream full-prompt SemIf scoring selects action type, vertical `UP/DOWN/NONE`, horizontal `LEFT/RIGHT/NONE`, and an optional duration. One to four readouts compose one allowed action. The original seven-type/51-label version became eight types/52 labels with D18.

Reason / original need: respect the provider's per-readout option limit and resolve observed direction-set errors while supporting every diagonal. Each axis receives the transcript and its initial held direction; no map, position or destination is included.

Impact: F04-F08; UC01-UC05; Adapter preserved, deterministic Command boundary unchanged. DecisionEngine.decide()/validate(), OpenJevAdapter.createChoices()/parseResponse() enforce the vocabulary. Historical model failures, eventual sampled successes and controlled bridge tests remain recorded in validation; this is not a universal accuracy claim.

D14 - Specify compatible native model deployment

Original: native model deployment and quantization were unspecified.

Implemented change: use pinned Q4_K_M GGUF weights/reference tokenizer. The Windows deployment uses a project-local overlay pairing the same-version official portable AVX2 CPU DLL with unchanged CUDA libraries; the installed dependency is preserved. Explicit GPU allocation extends the upstream CPU allocation setting while retaining its scoring.

Reason / original need: the initial CUDA wheel's CPU library required AVX512 unavailable on the verification CPU. GPU offload still needs compatible CPU instructions. Quantization makes local inference practical within the available memory.

Impact: F04-F08 runtime; UC01-UC05; Adapter boundary preserved. configure-semif-native.py `configure()`, server `load_scorer()`, SemIf setup document provenance and startup. Historical readiness/model checks passed with 34/34 layers offloaded; this is platform-specific evidence.

D15 - Adjust the decision deadline

Original: the initial implementation used a 1000 ms deadline including queue time.

Implemented change: default to 2000 ms while retaining finite provider timeouts, two active calls, one replaceable latest pending input and fail-closed current failures. Explicit launch options override environment settings.

Reason / original need: the initial four-partial burst expired at 1000 ms. A later pre-repeat burst returned the latest correction after 990.7 ms with the 2000 ms budget; the larger allowance accommodates queue/runtime variation without adding artificial delay.

Impact: F04/F05/F06/F08/F09; UC01-UC05; Adapter/Command unchanged. VoiceCommandAgent.interpret()/expire(), OpenJevAdapter.decide(), AppConfig.parse(), run.ps1 implement deadlines/configuration. Pre-repeat real CLI/burst checks passed; current repeat-schema GPU budget

verification remains outstanding.

D16 - Normalize correction word boundaries for axis readouts

Original: the same raw transcript representation reached every model readout.

Implemented change: preserve raw text in Java, the HTTP request and action-type scoring. Only after Qwen selects `SWITCH`, split commas/whitespace and join words with `,` for axis scoring.

Reason / original need: the unchanged ASR-like correction burst exposed sensitivity to unpunctuated text. Generic, idempotent formatting supplies word boundaries; it does not identify direction words or parse correction intent. Qwen still selects each axis.

Impact: F06, with F05/F07 regression coverage; UC03/UC04/UC05; Adapter preserved. DecisionEngine.decide(), bridge tests, LiveSemIfTest record the change. Twenty isolated cases and the original four-partial burst passed together before repeat support; wider language robustness remains unestablished.

D17 - Replace checkpoint recovery with start recovery

Original: F10/UC06/SD05 included checkpoint activation and respawn at the latest checkpoint. `Level`/`GameSnapshot` exposed progress, `resetToCheckpoint()` restored it, and views showed checkpoints.

Implemented change: remove checkpoint geometry, saved positions, activation events and progress displays. F10 becomes **Fall Detection and Start Recovery**, retaining its ID. Every fall restores `Level.start()` with zero velocity and no held inputs, while preserving cancellation/freshness and the fall/respawn lifecycle.

Reason / original need and scope: the user requested that every fall lose all progress, matching the intended rage-game experience. Recovery still allows another attempt; preserving intermediate progress is deliberately removed, not claimed as implemented.

Impact: F10 and F01/F09 feedback/cleanup; UC06; State, Observer, Command and MVC preserved. GameModel.evaluatePosition()/resetToStart(), Level.start(), FallingState.update(), RespawningState.update(), ApplicationController.onGameEvent(), GameView.onGameEvent() implement recovery. `GameSnapshot`/`GameEventType` no longer contain checkpoint data/events. Current UC06/SD05 show the replacement; 63-test, GUI and CLI regressions passed at this change.

D18 - Add structured repeat memory and refresh UML

Original: context contained held directions and prior accepted text, but no structured movement history. Seven action types/51 labels could not explicitly request replay after a timed movement expired.

Implemented change: add `MovementIntent(type, directions, durationMs)` for accepted PRESS/SWITCH/TIMED_PRESS and expose nullable `ControlContext.lastMovement`. Add empty-direction, zero-duration `REPEAT_LAST` as the eighth type/52nd label. Java creates new commands from remembered semantics; absent history produces neutral feedback. Permit one repeat per utterance, including across intervening partial interpretations.

Reason / original need: the user requested `again`. Resolved memory preserves exact duration/direction after expiry without asking the model to reconstruct an old action. Ordinary stop/expiry/keep/no-action preserve memory; pause/fall/restart/current failure clear it. This extends F05/F08/F09 rather than inventing another counted feature.

Impact: F05/F08/F09; UC03/UC05, with reset behavior in UC06; Adapter and Command preserved. MovementIntent, ControlContext, ControlManager.accept()/plan()/reset(), OpenJevAdapter.decide(), DecisionEngine.decide()/validate(), ApplicationController.acceptanceMessage() implement repeat. Restart an older running bridge to load the updated schema.

Historical evidence/artifacts: 73 Java tests, nine bridge tests, 20 unchanged baseline plus eight repeat/history CPU samples and packaged demo/live CLI checks passed. CPU samples used 60000 ms and live CPU CLI 30000 ms diagnostic caps, not the 2000 ms gameplay gate. At this stage original diagrams were archived under `docs/uml/stage1/` and 14 archived files were hash-checked while current diagrams were regenerated. Those archives/previews are now historical; current diagrams are UMLet SVGs.

D19 - Keep the CLI display in place while editing input

Original: full ASCII frames printed about once per second, scrolling the terminal while the user typed; Windows launched CLI through Gradle-forwarded streams.

Implemented change: use JLine 3.30.16 `Status` for a fixed bottom panel and `LineReader` for editable input. Resize the map, include the JNI terminal provider and launch the installed distribution directly after `installDist`. Quiet, redirected and unsupported-terminal sessions use plain nonperiodic state/event output.

Reason / original need: the user requested a stable game display that preserves partial commands. Direct startup exposes actual terminal capabilities. Renderers continue consuming shared snapshots; no second simulation or interpreter is added. This supersedes D06's scrolling interactive output.

Impact: F01/F04 presentation and F09/F10 feedback; UC02, with UC06 recovery display; MVC and Observer preserved. CliView.open()/readCommand()/refreshDashboard()/updateDashboard()/close(), Main.runCli(), build.gradle, run.ps1 implement the change. Seven CLI tests, full 76-test suite, packaged demo, automatic piped fallback, actual PowerShell quiet launch and native Windows ConPTY checks passed. Specific native Linux terminal interaction remains unverified.

UML impact: class/SD02 show terminal ownership, redraw, editable input, fallback and cleanup; SD05 distinguishes ordinary throttled refresh from lifecycle events. Earlier regeneration/checks are historical; current corresponding SVGs retain those responsibilities.

D20 - Add Linux launch and model-setup entry points

Original: recorded setup/launch automation was primarily PowerShell; portable Java source alone did not provide equivalent Linux bootstrap commands.

Implemented change: add run.sh, setup-semif.sh, run-semif.sh and setup-asr.sh. These locate the repository, honor documented options/environment and provide Java launch, explicit CPU/GPU model setup, and pinned ASR download/extraction. The ASR script uses `verify_archive()` for size/hash checks and an extraction-completion marker so failed extraction is repaired on rerun.

Reason / original need: support users building/running on Linux without translating Windows-specific paths, virtual-environment layout and native DLL handling. This is a supporting deployment change, not a new game feature or a rewrite of the Java application.

Impact: setup for F01-F10; UC01/UC02 deployment prerequisite; no pattern added or replaced. ASR setup, SemIf setup, AppConfig.parse() document configuration. Recorded Git Bash/fixture checks verify selected script control flow, including interrupted extraction; they do not establish native Linux inference, audio or terminal behavior. Consult validation for exact executed scope.

Preserved responsibilities and remaining completion work

The human chooses every direction and turn. AI receives control context, never level geometry or a destination, and returns only a bounded semantic proposal. Java owns schema/freshness validation, memory replay, cancellation, timing, simulation, fall detection and recovery. Both frontends share that core.

No design pattern has been replaced: MVC, Command, Adapter, Observer and State retain their roles with the refinements above. No checkpoint functionality is silently claimed after its removal. All feature videos and accessible timestamps remain pending. Specific microphone coverage and current default-budget GPU verification remain unverified; general user gameplay acceptance is recorded without extrapolating those details. Full Stage 3 testing/KUMA evaluation is a separate deliverable.

5. AI tools and collaboration log

The tool/model/task/accounting table and current human contribution below are part of the maintained collaboration log. Codex usage and subscription cost are student estimates, not measured platform exports. Local token and compute totals are identified as unavailable rather than assumed zero.

This is a representative Stage 2 record, updated for `Project-Stage2-Instruction.pdf` and the baseline project instructions. It records actual instructions, generated work, human decisions, and corrections. General gameplay acceptance and detailed source/test review are recorded separately.

Current status - October 27, 2026 (Toronto): the user reports having played the game and confirms that it works properly. The statement supports general gameplay acceptance; the interface, input mode, individual scenarios, and detailed code/test review were not specified. The latest recorded ordinary suite has **76 passing tests** from October 19; its saved results are being cited, not a new test execution. The pre-repeat 990.7 ms GPU burst remains historical evidence; the updated 2000 ms repeat-enabled burst has not been verified. The instructor accepted existing early commit history for weekly-progress evidence. Current report preparation and remaining evidence are tracked in validation. Earlier entries retain the evidence and pending status known when each interaction occurred.

Tools and models

Tool/platform	LLM/model	Features, classes, or development tasks	Total token cost/usage	Total money cost
OpenAI Codex, including delegated coding/review agents	GPT-6 model family, as identified by the session configuration; precise served snapshot unavailable	F01-F10; game/control core, JavaFX/CLI, controller/agent, adapters, concurrency review, tests, setup, diagrams, and implementation records	**Approximately 6 billion tokens** - student's estimate for project work, not a platform usage export	**Approximately CAD $282.50** - student-reported CAD $250 subscription charge multiplied by 1.13 for tax; project-specific allocation unmeasured
Local SemIf decision runtime	Qwen3.5 4B, Q4_K_M	Application interpretation for F04-F08, genuine model/agent checks, and packaged live CLI validation; `OpenJevAdapter` and `tools/semif/server.py`	**Unavailable** - cumulative local inference tokens were not recorded	No separately billed model API charge; local electricity and hardware costs were not measured
sherpa-onnx streaming recognition runtime	English streaming Zipformer; sherpa-onnx 1.13.5	F03; `SherpaOnnxAdapter`, streaming audio integration, and native sample-audio validation	**N/A for LLM tokens** - audio recognition, with no LLM token billing counter	No separately billed ASR API charge; local electricity and hardware costs were not measured

The October 27 Codex figures are student-supplied approximations. The 1.13 multiplier is treated as 13% tax, and CAD is assumed from the student's Toronto context; the billing period and whether the subscription also covered other work have not been specified. The CAD $282.50 figure is reported subscription spending, not a measured cost allocated solely to Say Swear. Confirm the currency, period, and estimate against available account records before submission. No private billing documents or account identifiers are included here. Local model token totals and compute costs cannot be inferred from test counts.

The AI models running **inside Say Swear** are distinct from the development assistant. The user selected **Qwen3.5 4B** and identified the intended backend as **SemIf**. The assistant then researched openjev.com and TheoLeeCJ/SemIf-OpenJev. Recorded checks cover native sample-audio ASR, genuine model cases, an earlier four-partial agent burst, and packaged live CLI interaction. Their dates, diagnostic budgets, historical failures, and remaining evidence limits appear in validation.

October 20, 2026 - Authorized report update and gameplay acceptance

- **Task:** Bring Stage 2 documentation and the implementation report into line with the new Stage 2 instructions.
- **Input/Instruction:** The user authorized report/documentation work and stated, "I also played the game already so I know it works properly."
- **AI Contribution:** Audited the new requirements; separated existing implementation evidence from missing report, video, usage/cost, weekly-activity, and review records; prepared updated documentation and identified the remaining review items.
- **Human Contribution:** Reported personal gameplay experience and general acceptance that the game works properly. The user did not identify a particular frontend, microphone session, scenario coverage, or completed source-code review in this statement.
- **Outcome:** General gameplay acceptance is recorded. Existing test evidence remains unchanged. At the time, detailed review items, feature-video links, usage/cost totals, and weekly-progress evidence remained pending. The instructor's October 27 response about the early commit history is recorded in validation. This documentation update did not claim new application or model test results.

Representative implementation interactions

Interaction	Human contribution	AI contribution and actual result
C01 - Shared Java design	Specified small scope, both frontends, and human navigation; authorized implementation.	Implemented game/control, adapters, frontends, and controller coordination; recorded build and integration checks with their actual scope.
C02/C08-C11 - Correct and validate the backend	Selected Qwen3.5 4B and clarified that the intended backend was SemIf.	Corrected the earlier provider assumption, isolated native-runtime failures, factored bounded model choices, and recorded real inference/CLI cases separately from deterministic checks.
C04/C09 - Freshness and deadlines	Required live correction and interruption without stale actions.	Added epoch/version/revision validation, cancellation, bounded requests, and queue-inclusive expiry. Historical burst timing is preserved without claiming current repeat-enabled performance.
C12 - Start recovery	Rejected checkpoints because they conflicted with the rage-game concept.	Removed checkpoint progression and changed F10, state transitions, tests, views, and traceability to neutral start recovery.
C13 - Start marker	Requested a red arrow with white YOU ARE HERE text.	Added the presentation cue at the neutral starting position; no additional gameplay feature was counted.
C14 - Repeat after timer expiry	Identified that replaying the latest release action could make `again` ineffective; requested repeat support and diagram readability fixes.	Introduced `MovementIntent` and `REPEAT_LAST`; retained movement history after expiry, prevented duplicate same-utterance replay, and recorded repeat checks and remaining timing limits.
C15-C16 - Terminal display and diagrams	Reported scrolling CLI output and requested diagram consistency.	Added JLine in-place display and editable input, direct launch, plain fallback, terminal tests, and corresponding diagram changes.
C17 - Linux ASR setup	Identified the missing Linux setup script and requested it.	Added pinned/resumable/verified setup with fixture checks; recorded that native Linux recognition was not exercised.

These representative interactions document actual scope decisions, corrections, generated work, and checks. The full log retains the original Task, Input/Instruction, AI Contribution, Human Contribution, and Outcome fields for C01-C17. The user's current gameplay acceptance is recorded above; it does not imply that every generated source file or test has received detailed human review.

6. Development and verification summary

Current acceptance and weekly activity

This record separates executed checks from outstanding evidence, as of **October 27, 2026 (Toronto)**. Earlier development checks were recorded on September 30; later verification and setup work are dated below. The latest saved ordinary Java results are from **October 19, 2026: 76 tests passed, with zero failures, errors, or skipped tests**. This report update cites those saved results and does not represent another execution. The feature mapping identifies the corresponding implementation files.

Current acceptance and verification status

- **General gameplay accepted:** on October 20, the user stated, "I also played the game already so I know it works properly." This is user-reported gameplay acceptance. The frontend, input device, scenarios exercised, and detailed source/test review were not specified.
- **Ordinary Java verification:** the October 19 saved suite records 76 passing tests. Historical 63/73-test runs below describe earlier revisions.
- **Real-model timing:** the **990.7 ms** GPU correction burst below predates repeat support. The updated repeat-enabled **2000 ms** GPU burst gate remains unverified. Repeat CPU cases used longer diagnostic deadlines, as recorded below.
- **Evidence still needed:** a completed feature-video index and detailed code-understanding review. Student-estimated AI token/subscription figures are now recorded in the collaboration log, with currency, period, and allocation still to confirm. The instructor accepted the existing early commit history for weekly-progress evidence, as recorded below. Specific microphone/device-failure and platform coverage are not established by the general gameplay statement. These evidence gaps do not require repeating the already accepted general play session.

Weekly code-activity summary

The written Stage 2 instructions set a baseline of **five meaningful implementation/test commits pushed during each Toronto-dated reporting period**. On October 27, the student supplied the instructor's reply to a direct question about substantial implementation completed before October 20 and the weekly count: the existing history works. The screenshot remains outside the repository because it contains names and account details. This report presents the actual history as the accepted evidence; it does not assign early commits to later weeks or claim that each week's numerical baseline was met.

Period	Dates, inclusive (2026, Toronto)	Written baseline	Actual code/test pushes as of October 27	Progress evidence
1	October 20-26	5	0	Existing earlier implementation and test history accepted by instructor.
2	October 27-November 2	5	0 so far	Existing earlier history accepted; add any later genuine work at submission.
3	November 3-9	5	Not started	Existing earlier history accepted; record actual activity after this period.

The early Stage 2 evidence includes the shared game, agent, GUI/CLI implementation and tests (September 30), Linux application and SemIf launchers (October 6), and the integration into public main with Linux ASR setup (October 19). These are cited at their real dates. Report-only edits and artificial subdivisions are not represented as code progress.

Executed verification and limits

Check	Recorded outcome	What it establishes / limit
Ordinary Java suite and distribution	Latest saved October 19 suite: 76 passing tests, zero failures/errors/skips; build/installDist recorded as passing.	Domain/control, freshness/cancellation, agent scheduling, mock HTTP, capture lifecycle, and CLI behavior. Does not run a real LLM or microphone.
SemIf bridge contracts	Nine controlled-score tests passed after repeat support.	Request/schema/action composition and structured memory. Does not measure real-model language accuracy.
Native ASR sample	Real 6.63-second WAV yielded 17 changing partials and one endpoint, expected phrases, zero errors.	Actual sherpa-onnx native/model streaming recognition; no microphone was opened.
Earlier genuine-model checks	Twenty baseline cases passed; isolated requests 242.2-503.1 ms with a 15-second diagnostic deadline; four-partial burst returned latest LEFT correction after 990.7 ms within 2000 ms.	Historical pre-repeat semantic/agent/CLI evidence. Not a measurement of current repeat-enabled or microphone latency.
Repeat-enabled CPU checks	Twenty baseline plus eight repeat/history cases passed with a 60000 ms diagnostic cap; packaged real CLI used 30000 ms.	Genuine Java/SemIf/Qwen repeat semantics and execution. Does not establish the default 2000 ms performance gate.
GUI and packaged CLI	Recorded JavaFX text-entry, movement, Stop & release, restart, packaged real/demo CLI, native Windows terminal, and piped fallback checks passed.	Actual application wiring and terminal behavior; demo and genuine-model results remain distinguished.
Linux ASR setup	Bash/GNU fixture checks passed on Windows: fresh/cached, resume, invalid archive, unsafe path, and interrupted extraction cases.	Downloader/control flow. Native Linux recognition, GUI, and interactive terminal behavior remain unverified.
User gameplay	User reports having played the game and that it works properly.	General gameplay acceptance. Frontend, microphone mode, command coverage, detailed source review, and measured latency were not specified.

Full commands, earlier failures, correction history, counts, diagnostic settings, and limits remain in the validation record. No application/model tests were rerun merely to prepare this report. Report generation checks document structure, links, diagram embedding, and rendered page layout separately from application verification.

Important defects found and corrected

- **Incorrect provider identity:** replaced the unrelated hosted-service assumption with the user-selected local SemIf/Qwen runtime behind the same Java interface.
- **Native inference startup:** isolated an incompatible native instruction requirement and documented pinned compatible runtime setup without changing the application core.
- **Model direction/correction failures:** factored action/axes/duration readouts and used generic word-boundary formatting only for model-selected SWITCH axes; Qwen still chooses intent and directions.
- **Burst deadline failure:** increased the original queue-inclusive 1000 ms budget to 2000 ms after the unchanged correction burst expired; the historical pass and outstanding current repeat-enabled check remain distinct.
- **Repeat after automatic release:** introduced structured movement memory so expiry does not replace remembered movement with release, and same-utterance partials cannot rearm it repeatedly.

- **Scrolling CLI input:** replaced repeated full-screen printing with a fixed game panel and separate editable prompt, while retaining plain output for redirected/unsupported terminals.
- **Interrupted ASR extraction:** added a verified completion marker so a failed extraction is repaired rather than treating truncated files as ready.

Focused human review and remaining submission work

The completed gameplay session is already accepted. Detailed source/test review has not yet been reported complete. Record its actual scope, corrections, and outcome in the collaboration log, including understanding of the shared core, the AI/execution boundary, freshness/cancellation, timer/repeat semantics, lifecycle recovery, and five pattern roles.

Feature video URLs/timestamps remain **PENDING**. The AI accounting section records the student's approximate token and subscription figures with its unmeasured categories and items to confirm. The instructor accepted the existing early commit history as weekly-progress evidence; section 6 gives its actual dates and links. Specific microphone/recovery coverage, native Linux runtime coverage, and repeat-enabled performance under the default 2000 ms deadline remain unverified. Sections 5-7 retain these gaps explicitly without inferring them from general gameplay acceptance.

Stage 2 requires basic implementation verification. Full deterministic/KUMA evaluation remains a later-stage requirement and is not represented as completed or included in this public Stage 2 report.

7. Demonstration index

Status: PENDING USER RECORDINGS. No video link or timestamp has been supplied. The user has played the game and reports that it works properly; this is gameplay acceptance, not a substitute for the rubric's feature videos. Fill every row and regenerate `stage2-report.pdf` before submission.

One recording can cover several features. Give each segment a visible feature ID/name, show the actual input and running result, briefly explain the AI role, and include a relevant alternative/error when practical. Collectively show the JavaFX GUI and the Java CLI. For model-dependent demonstrations, run the genuine local SemIf/Qwen service with demo mode disabled. F02/F03 need actual microphone capture and changing transcripts; typed input bypasses them.

Feature	Interface and actual demonstration	Result / alternative to show	Video URL	Time (start-end)
F01 - Real-Time Precision Movement	GUI and CLI: issue a direction and inspect the shared position, velocity, held set, and lifecycle.	Continuous movement; neutral state after stop. Show GUI level and CLI map/state.	PENDING	PENDING
F02 - Continuous Voice-Control Session	GUI: Start listening, speak successive instructions without restarting capture, then Stop & release.	Visible readiness/listening/stopped status; capture and movement stop. If capture is unavailable, show the error rather than claiming a successful session.	PENDING	PENDING
F03 - Streaming Speech Transcription	GUI microphone: speak an instruction and correction while the transcript is visible.	Changing partial text before the utterance finishes; continued capture for a new utterance. Explain that ASR makes text, and SemIf interprets that text separately.	PENDING	PENDING
F04 - Natural-Language Movement Interpretation	CLI with genuine model: enter `right`, `go up`, and `stop`; inspect accepted-action messages and resulting state.	Model-selected bounded actions become deterministic movement. Include an unrelated instruction and explain `NO_ACTION` preserves existing movement; use `/pause` to stop immediately.	PENDING	PENDING
F05 - Context-Aware Commands	GUI or CLI: hold right, request `other way`; try `keep going`; after a timed action ends, request `again`.	Reversal from held context, continuation without replay, and replay from stored movement. After restart, `again` gives the no-history message.	PENDING	PENDING
F06 - Live Command Correction / Interruption	GUI speech or CLI text: correct active movement with `right - no, left`, then another instruction.	New valid instruction replaces movement; earlier held directions/timers do not reappear. Explain the epoch/version/revision checks from code; a video alone cannot prove arbitrary response ordering.	PENDING	PENDING
F07 - Combined Directional Commands	GUI or CLI: enter `up and right` from a suitable safe position.	Two perpendicular held directions and diagonal movement; explain normalized speed. Follow with a single direction or stop to show replacement.	PENDING	PENDING
F08 - Magnitude / Timed Commands	GUI or CLI: request `a little right`, wait for release, then say/type a fresh `again`.	A 200 ms hold expires automatically; repeat starts a fresh full-duration hold even though the held set was empty.	PENDING	PENDING

Feature	Interface and actual demonstration	Result / alternative to show	Video URL	Time (start-end)
F09 - Control-State and Conflict Management	GUI Stop & release or CLI `/pause` during movement, then restart and try `again`.	Immediate neutral controls, cleared repeat memory, and no old movement resuming. Briefly show the relevant freshness/cancellation code or test as supporting evidence alongside the running app.	PENDING	PENDING
F10 - Fall Detection and Start Recovery	GUI or CLI: deliberately leave the safe path after making progress.	Fall and return to the level start with zero velocity/held inputs; `again` has no history. If demonstrating voice, show requested capture recovering under a fresh session or report the actual failure.	PENDING	PENDING

Separate acceptance criteria for F02, F03, and F04

- **F02 - capture/session:** one start maintains successive audio input, visible status reflects readiness, and stopping releases capture and controls. Correct recognition is not necessary to establish the capture lifecycle itself.
- **F03 - transcription:** audio produces changing visible partial text before recording completion and distinct utterances at endpoints. Movement interpretation is not necessary to establish incremental transcription.
- **F04 - interpretation:** typed natural language, without a microphone, reaches real SemIf/Qwen and produces a bounded accepted action followed by actual game movement or stop. ASR correctness is independent of this criterion.

Before marking videos complete

- Replace every PENDING URL and timestamp with actual evidence. Use timestamped links for segments in a longer recording.
- Check that the instructor and teaching assistants can open links without an access request.
- Keep feature IDs/names visible or stated, show the input and resulting application behavior, and distinguish model inference from deterministic execution.
- State the actual timeout/runtime settings used. A slower diagnostic run is not evidence that the default 2000 ms deadline passes.
- Do not expose credentials, personal filesystem paths, private repository content, or unrelated windows in the recordings.
- Update the feature mapping and regenerate the PDF after adding the links. Review the final PDF's links as well as this index.

Diagram appendices

The following pages embed the supplied UMLet SVGs as vectors without changing the diagram files. Pages use larger formats to preserve readable text at normal zoom. Source links are also indexed in section 3. These diagrams explain design and runtime contracts; they do not replace the required running-application videos.

Appendix A - UML class diagram

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/class-diagram.svg

Say Swear - Stage 1 revised Java design
Read top to bottom within each package. Key methods are shown; concrete classes inherit interface operations.
One selected GUI/CLI shares the controller, game, controls and agent. AI receives control context only; the human chooses the route.
Typed attributes/results also express value dependencies. CompletionStage abbreviates CompletionStage<ActionDecision>.

Presentation / MVC

«interface»
GameView
+render(snapshot: GameSnapshot)
+showMessage(message: String)
+showTranscript(text: String)
+showVoiceStatus(status: String)
+onGameEvent(event: GameEvent)

JavaFxGameView
+root(): Parent
+bindController(controller)
+requestStartVoice()
+requestStopVoice()
+render(snapshot: GameSnapshot)

CliView
-terminal: Terminal [0..1]
-status: Status [0..1]
-lineReader: DashboardReader [0..1]
-latestSnapshot: GameSnapshot
+open(quietCli: boolean): CliView
+setSessionDescription(value: String)
+showHelp()
+readCommand(): String
+setInteractive(interactive: boolean)
+render(snapshot: GameSnapshot)
+onGameEvent(event: GameEvent)
+close()
-refreshDashboard()
-updateDashboard()

Views implement GameView; GameView extends GameObserver.
CliView plans a JLine status panel and editable prompt, with a plain-terminal fallback.
Terminal, Status, Parent and CompletionStage are library types.
DashboardReader is a private CliView helper.

Application / MVC and Observer

«interface»
GameObserver
+onGameEvent(event: GameEvent)

Main
+main(arguments: String[])
-runCli(configuration)

ApplicationController
-voiceRequested: boolean
-version, utterance: long
+start()
+submitTextCommand(text: String)
+handleTranscript(text, sourceUtterance, epoch)
+handleDecision(decision: ActionDecision)
+startVoiceControl()
+stopVoiceControl()
+handleVoiceReady(epoch)
+handleVoiceFailure(message, epoch)
+pauseControls()
+restart()
+onGameEvent(event: GameEvent)
+getSnapshot(): GameSnapshot
+close()
-submit(text, inputUtterance)
-advance(seconds: double)
-isCurrent(update: TranscriptUpdate): boolean
-failCurrent(message: String)
-invalidateControls()
-publish()

Language interpretation / Adapter

VoiceCommandAgent
-baseContext: ControlContext
+interpret(update, context): CompletionStage
+reset()
+close()

«interface»
DecisionModel
+decide(update, baseContext): CompletionStage
+close()

OpenJevAdapter
+decide(update, baseContext): CompletionStage
+close()

DemoDecisionModel
+decide(update, baseContext): CompletionStage

loopback HTTP

«external HTTP service»
Local SemIf / Qwen3.5 4B
POST /decide
GET /health

Deterministic controls / Command

ControlManager
-previousCommand: String
-lastAccepted: ActionDecision
-lastMovement: MovementIntent [0..1]
-lastRepeatedUtterance: long
-epoch, revision, latestVersion: long
+beginInput(version: long)
+getContext(): ControlContext
+accept(decision: ActionDecision): boolean
+validate(decision: ActionDecision): boolean
+plan(decision: ActionDecision): List<GameCommand>
+advance(deltaTime: double)
+reset()

ActionExecutor
-activeTimed: List<TimedPressCommand>
+execute(commands: List<GameCommand>)
+cancelAll()
+advance(deltaTime: double): boolean

«interface»
GameCommand
+execute(game: GameModel)
+cancel(game: GameModel)

KeyDownCommand
-direction: Direction

KeyUpCommand
-direction: Direction

ReleaseAllCommand

TimedPressCommand
-directions: Set<Direction>
-durationMs: int
-remainingMs: double
+advance(deltaTime, game): boolean

Game model

GameModel
-heldDirections: Set<Direction>
-observers: List<GameObserver>
+update(deltaTime: double)
+evaluatePosition()
+press(direction: Direction)
+release(direction: Direction)
+releaseAll()
+resetToStart()
+restart()
+getSnapshot(): GameSnapshot
+addObserver(observer: GameObserver)
+notifyObservers(event: GameEvent)

Player
-position, velocity: Vector2
-state: PlayerState
+update(game, deltaTime)
+move(deltaTime, held)
+setState(state: PlayerState)
+respawn(position: Vector2)

Level
-safePath: List<Vector2>
-pathHalfWidth: double
-goal: Vector2
+start(): Vector2
+isSafe(position: Vector2): boolean
+isGoal(position: Vector2): boolean

GameModel is the command receiver and observer subject.
One serialized application loop owns all mutable game/control state.
Each fall cancels controls and restores Level.start(); there are no checkpoints.

Voice input / Adapter

VoiceInputService
+start(epoch: long)
+acceptAudio(samples: float[])
+onPartial(text: String)
+onEndpoint()
-partial(session, text: String)
-endpoint(session)
+stop()
+close()

«interface»
StreamingSpeechRecognizer
+start(onPartial, onEndpoint, onError)
+acceptAudio(samples: float[])
+stop()
+close()

MicrophoneDevice
+open()
+readSamples(): float[]
+close()

SherpaOnnxAdapter
+start(onPartial, onEndpoint, onError)
+acceptAudio(samples: float[])
+stop()

Player lifecycle / State

«interface»
PlayerState
+update(player, game, deltaTime)
+allowsMovement(): boolean
+status(): PlayerStatus

ActiveState

FallingState

RespawningState

FinishedState

Immutable interpretation values

«record»
TranscriptUpdate
text: String
utteranceId, version, epoch: long

«record»
ControlContext
heldDirections: Set<Direction>
previousCommand: String
epoch, revision, latestVersion: long
movementAllowed: boolean
lastMovement: MovementIntent [0..1]

«record»
MovementIntent
type: ActionType
directions: Set<Direction>
durationMs: int

Memory contains resolved movement semantics.
Timer expiry and spoken stop preserve it.
Pause, fall, restart and current errors clear it.
REPEAT_LAST creates fresh commands, once per utterance.

«record»
ActionDecision
type: ActionType
directions: Set<Direction>
durationMs: int
transcriptVersion, utteranceId: long
epoch, contextRevision: long
sourceText: String
+withRequest(update, context): ActionDecision

«enumeration»
ActionType
PRESS
RELEASE
SWITCH
TIMED_PRESS
KEEP_CURRENT
RELEASE_ALL
NO_ACTION
REPEAT_LAST

«enumeration»
Direction
UP
DOWN
LEFT
RIGHT

ActionDecision carries request identity for stale-result rejection. MovementIntent deliberately contains no request identity or running timer.
Values listed by name in attributes and method signatures are shared between the packages above.

Game values

«record»
GameSnapshot
position, velocity: Vector2
heldDirections: Set<Direction>
status: PlayerStatus
level: Level

«record»
GameEvent
type: GameEventType
snapshot: GameSnapshot

«record»
Vector2
x: double
y: double

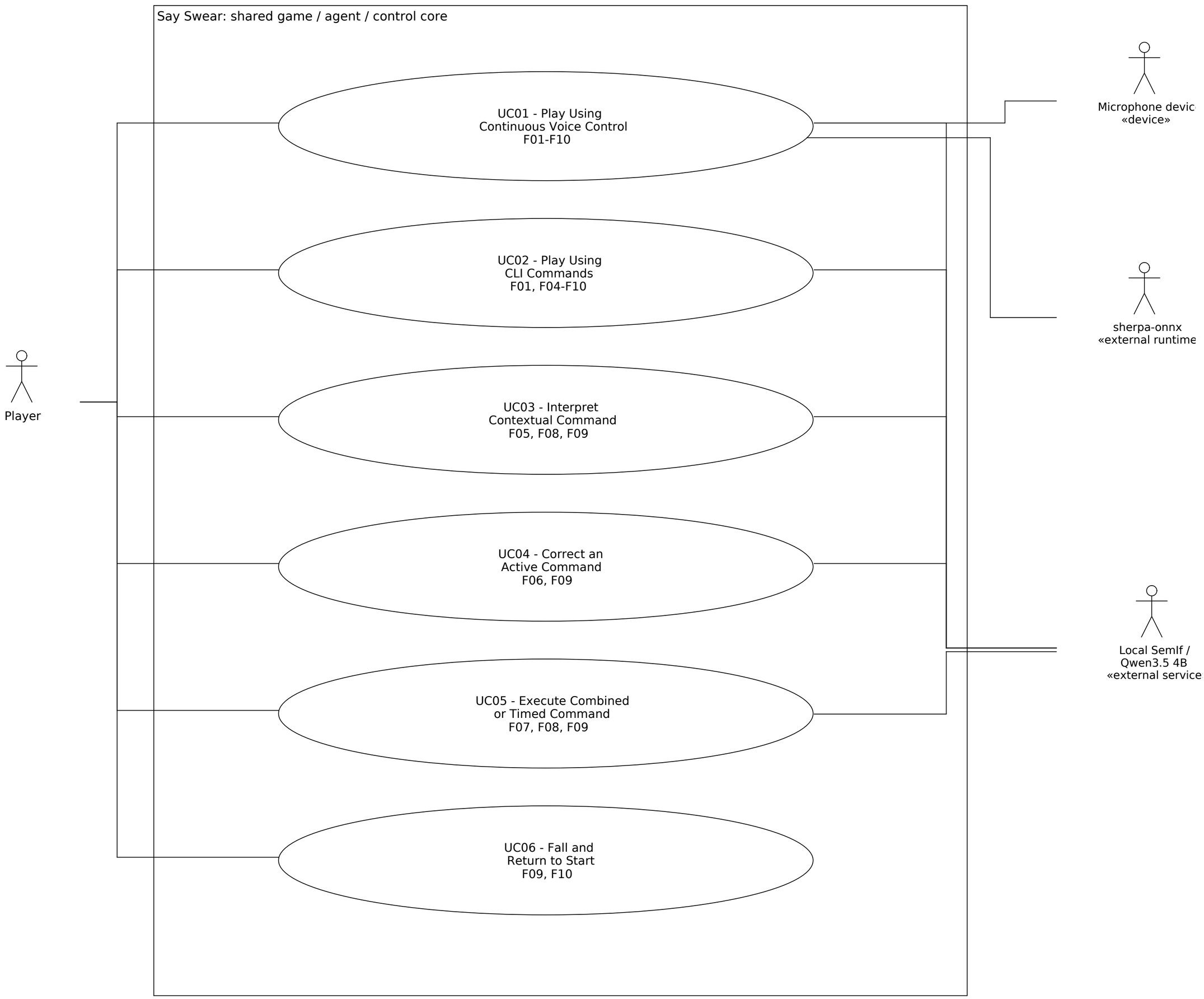
«enumeration»
PlayerStatus
ACTIVE
FALLING
RESPAWNING
FINISHED

«enumeration»
GameEventType
STATE_CHANGED
FALL_DETECTED
RESPAWNED
GOAL_REACHED

Appendix B - Use-case diagram

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/use-case-diagram.svg

Say Swear - Stage 1 use cases
One Java application, launched with either JavaFX or a CLI frontend. UC03-UC06 occur during UC01 or UC02.
The human selects every direction; no actor or use case delegates route planning to the model.

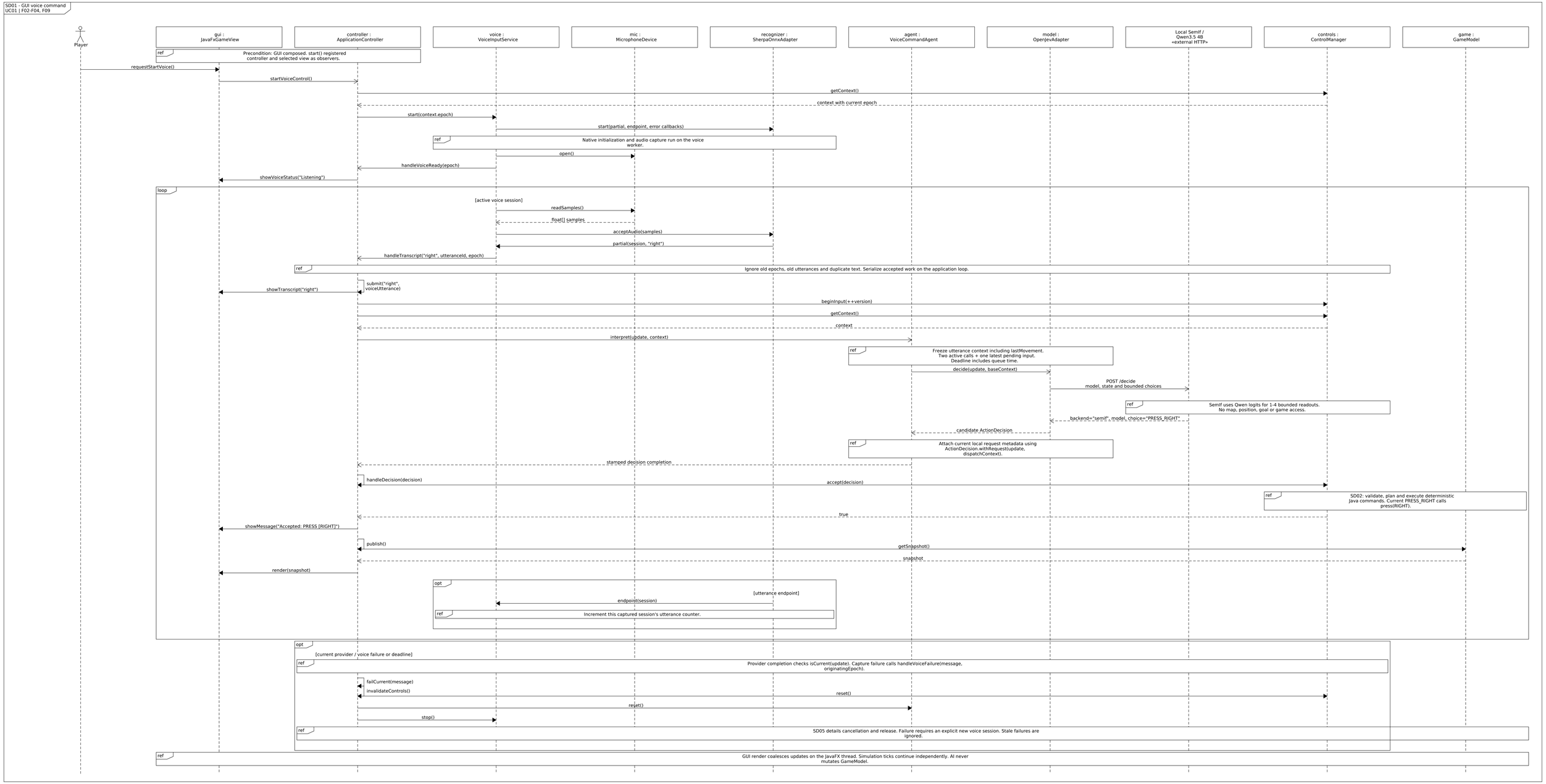


UC03 context: "other way" uses held directions;
"keep going" preserves control; "again" replays stored
MovementIntent, including a new timer after expiry.
UC04 replaces an active instruction with its newer correction.
UC05 supports diagonal or bounded-duration movement.

UC06 is triggered by leaving the safe path.
It is a system response experienced by the Player.
Java cancels commands, clears memory and returns to Level.start().
UC02 replaces microphone/transcription with typed instructions.
Actor associations show participation, not message order.

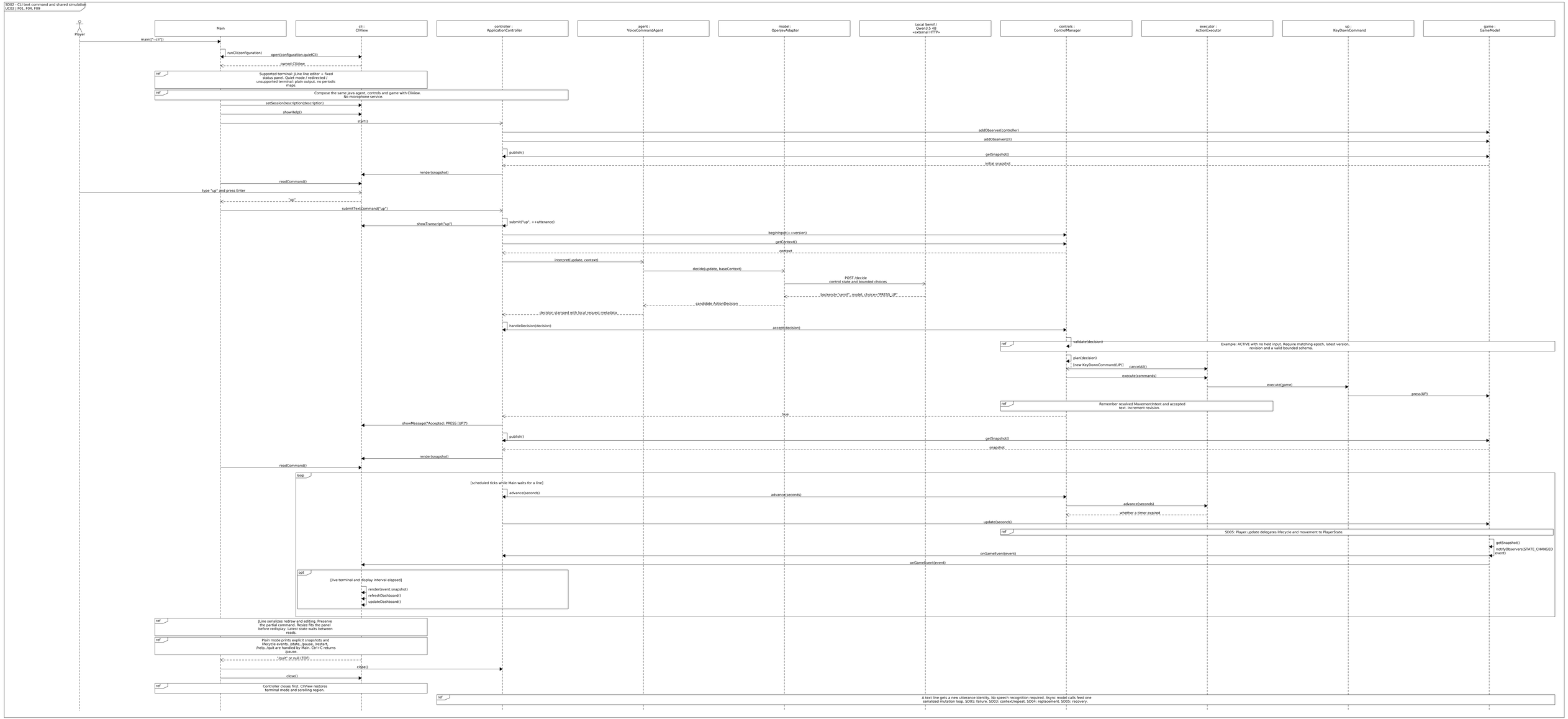
Appendix C - SD01 GUI voice command

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/sequence-diagrams/sd01-gui-voice-command.svg



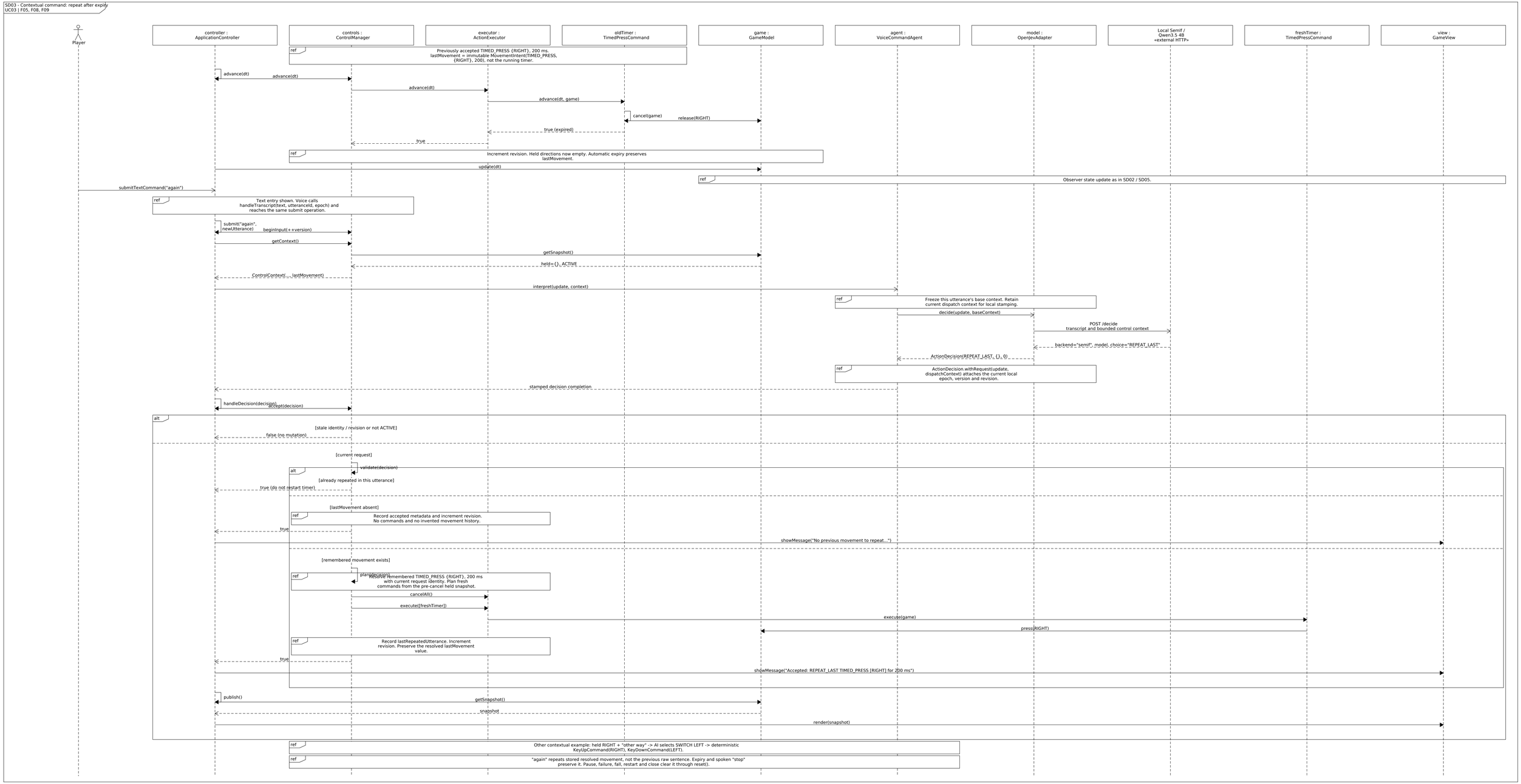
Appendix D - SD02 CLI text command

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/sequence-diagrams/sd02-cli-text-command.svg



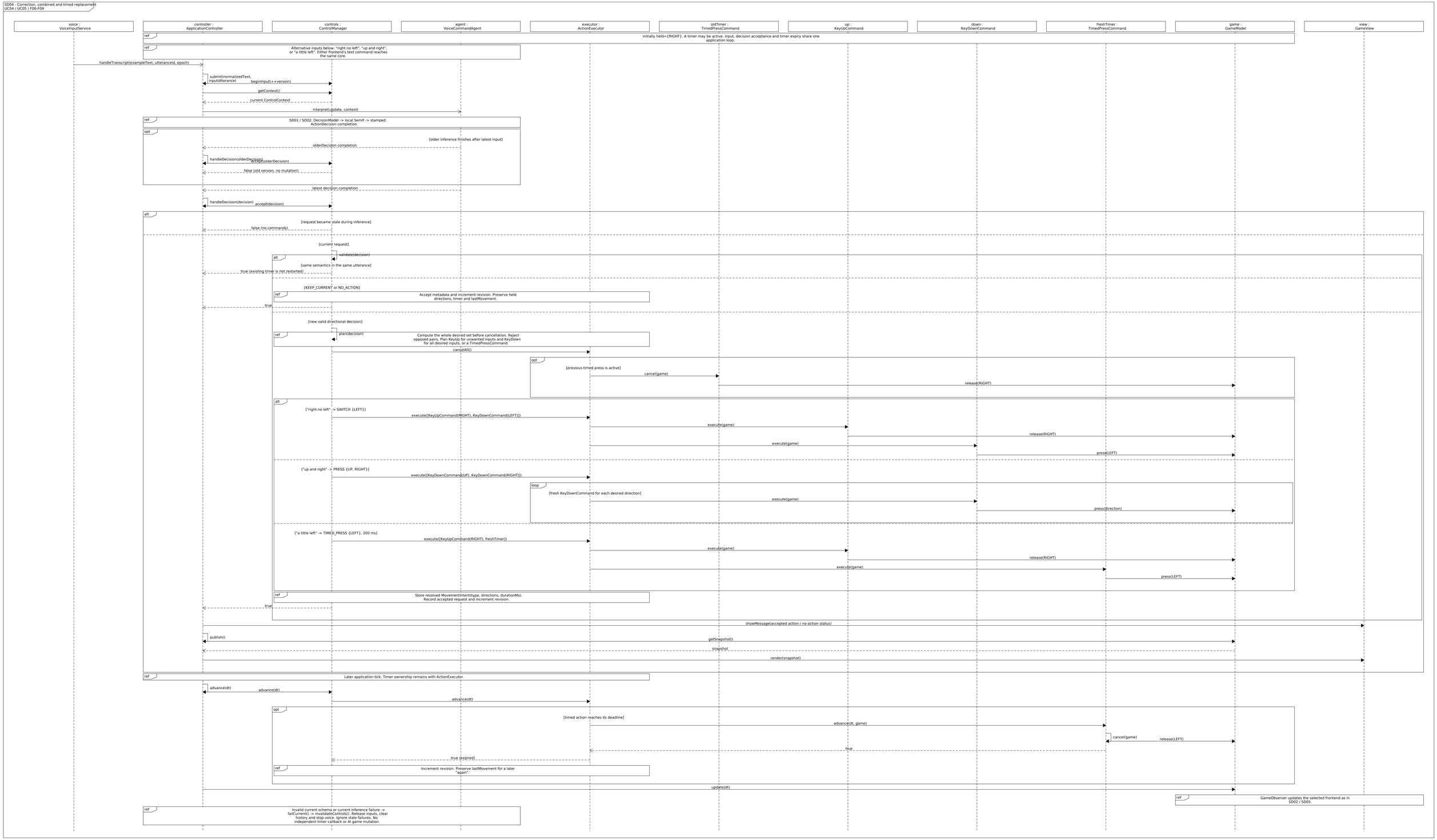
Appendix E - SD03 contextual command and repeat

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/sequence-diagrams/sd03-contextual-command.svg



Appendix F - SD04 correction, combined and timed commands

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/sequence-diagrams/sd04-live-correction.svg



Appendix G - SD05 fall and start recovery

Existing UMLet SVG | large-format vector page | zoom to inspect. Source: docs/uml/sequence-diagrams/sd05-fall-recovery.svg

